

Instituto Superior de Engenharia

Politécnico de Coimbra

P5

Meta 2

Ricardo Duarte Rodrigues | 2022147797
a2022147797@isec.pt
Diogo Moreira Albuquerque | 2021143818
a2021143818@isec.pt

Índice

1. Introdução 3
2. Arquitetura Geral 4
3. Mecanismos de Sincronização 4
4. Estruturas de Dados 5
   - 4.1. Struct ACAO 5
   - 4.2. Struct BOLSA 5
   - 4.3. Struct DATA_FUNCOES 5
   - 4.4. Struct DATA_THREAD_BOARD 6
   - 4.5. Struct CLIENTE 6
   - 4.6. Struct UTILIZADORES 6
   - 4.7. Struct DATA_THR_CLIENTE 7
5. Board 7
6. Bolsa 7
7. Cliente 8
8. Tabela de Funcionalidades Implementadas 9
9. Conclusão 10

1. Introdução

Este trabalho prático visa a implementação de uma bolsa de valores simplificada, composta por vários programas que simulam o funcionamento de um mercado de ações. O objetivo é explorar os conceitos e a matéria dada na disciplina de Sistemas Operativos II, sem a necessidade de um alto grau de realismo. O objetivo deste trabalho será implementar os principais conceitos de uma bolsa de valores, simular o funcionamento de um mercado de ações, explorar a comunicação entre processos e desenvolver uma interface para o cliente.

Para isto iremos desenvolver 4 programas, o programa bolsa que tem a função de regular toda a gestão de dados e de operações, o programa cliente desempenha o papel de concretizar a interface de utilizador entre o sistema bolsa e o utilizador comum, o programa da board, que apresenta ao utilizador as informações atualizadas da bolsa e o programa boadUI, sendo este opcional, tendo a sua função idêntica ao programa da board na medida em que apenas apresenta informação. Porém este programa terá a interface é gráfica e a informação é apresentada como um gráfico de barras.

2. Arquitetura Geral

A aplicação Bolsa é a responsável por gerir todo o funcionamento da bolsa de valores. Sem esta, nem os clientes nem a board conseguem funcionar. Deste modo, esta deve ser sempre aberta antes dos outros.

A bolsa vai ser a responsável por todas as vendas e compras de ações, consequentemente irá ser a responsável também pela variação dos preços destas.

O cliente irá interagir com a bolsa, e para este realizar a compra e venda de ações terá ainda de efetuar o login para que possa realizar as operações que desejar.

Já a board irá interagir com a bolsa com a única função sendo apresentar o nome, valor e quantidades as ações disponíveis no mercado de ações.

3. Mecanismos de Sincronização

Para garantir a distribuição específica de informações do cliente (Bolsa-» Cliente), utilizamos named pipes dedicado ao cliente, através dos quais a informação desejada é transmitida.

A informação sobre as várias ações das empresas (Bolsa-» Board) é compartilhada por meio de memória partilhada, onde a bolsa atualiza os valores das ações e partilha-os com a memória partilhada. Esta informação é partilhada com o board, que lê e mostra o seu conteúdo. Para garantir a sincronização correta entre processos diferentes, é utilizado um mutex.

No paradigma de comunicação entre a bolsa e o cliente, a bolsa cria uma thread para a instância que se conecta ao named pipe. Essa thread é responsável pela leitura e escrita de dados, assegurando uma comunicação eficiente. Para garantir a atualização contínua e correta dos dados sobre as ações das empresas, a bolsa utiliza mecanismos de sincronização, como eventos, garantindo que as informações exibidas pelo board estejam sempre atualizadas.

Além disso, eventos e mutex são utilizados para coordenar ações entre os processos e as threads, garantindo que as operações ocorram de maneira ordenada e eficiente, evitando conflitos e melhorando a performance do sistema como um todo.

4. Estruturas de Dados

4.1. Struct ACAO

```
typedef struct {
        TCHAR nome[MAX_NOME];
        int preco;
        int quantidade;
} ACAO;
```

Estrutura responsável pelas ações.

4.2. Struct BOLSA

```
typedef struct {
        ACAO* acoes;
        int n_acoes_bolsa;
}BOLSA;
```

Estrutura responsável pelos dados da bolsa.

4.3. Struct DATA_FUNCOES

```
typedef struct {
        BOLSA* b;
        HANDLE* hMutex, hPipe, hEv_atualizaBoard;
        BOOL* continua;
}DATA_FUNCOES;
```

Estrutura responsável pela variação dos preços das ações no recorrer do programa.

4.4. Struct DATA_THREAD_BOARD

```
typedef struct {
        BOLSA* bolsa;
        HANDLE* hEv_atualizaBoard, hEv_avisaThrBoard;
        TCHAR* lpMapAddress;
        BOOL* continua;
}DATA_THREAD_BOARD;
```

Estrutura responsável pela thread que gere a board.

4.5. Struct CLIENTE

```
typedef struct {
        TCHAR nome_cliente[MAX_NOME];
        TCHAR passe[5];
        DOUBLE saldo;
        BOLSA* carteira_acoes;
}CLIENTE;
```

Estrutura responsável pelos dados dos clientes que irão utilizar a bolsa de valores.

4.6. Struct UTILIZADORES

```
typedef struct {
        CLIENTE* clientes;
        int n_utilizadores;
}UTILIZADORES;
```

Estrutura responsável pela contabilização do número de clientes na bolsa.

4.7. Struct DATA_THR_CLIENTE

```
typedef struct {
        BOLSA* bolsa;
        UTILIZADORES* utilizadores;
}DATA_THR_CLIENTE;
```

Estrutura responsável por armazenar as informações dos utilizadores e da bolsa para gerir a thread para cada cliente.

5. Board

Este programa apresenta as informações relativamente a bolsa de valores e apenas deve disponibilizar ao utilizador uma forma de proceder ao seu encerramento.

6. Bolsa

Este programa permitirá acrescentar empresas através do comando “addc”, não podendo passar um numero máximo de 30 empresas; Listar todas as empresas com o comando “listc”, mostrando o seu nome, número de ações disponíveis e peço atual das ações; Redefinir o custo das ações de uma determinada empresa com o comando “stock”, devendo indicar o nome da empresa e o novo preço de cada ação; Listar utilizadores através do comando “users”, permitindo listar todos os utilizadores registados, mostrando o seu username, saldo atual e estado; Pausar as operações de compra e venda pelo comando “pause”, faz com que as operações de compra e venda sejam suspensas durante um período de tempo; Encerrar o sistema através do comando “close”.

7. Cliente

Autenticar um utilizador pelo meio do comando “login”; Listar todas as empresas com o comando “listc”, listando assim todas as empresas existentes na bolsa, mostrando o seu nome, número de ações disponíveis e preço atual das ações; Comprar ações através do comando “buy”, colocando uma operação de compra de um número de ações especificado de uma determinada empresa. Caso não exista o número de ações solicitado, a operação não deve ser concretizada, sendo o utilizador informado desse facto; Vender ações por meio do comando “sell”, metendo uma operação de venda de um número de ações especificado de uma determinada empresa. Se o utilizador não tiver essa quantidade de ações, a operação é rejeitada; Consultar o saldo através do comando “balance”; Sair da aplicação através do comando “exit”.

8. Tabela de Funcionalidades Implementadas

ID	Descrição da Funcionalidade / Requisito	Estado
0	Variação de preço	Implementada
1	Venda de ações	Implementada (com erros de execução)
2	Compra de ações	Implementada (com erros de execução)
3	Acrescentar empresas	Implementada (com erros de execução)
4	Listar empresas	Implementada (com erros de execução)
5	Redefinir os custos das ações	Implementada (com erros de execução)
6	Listar utilizadores	Implementada (com erros de execução)
7	Pausar as operações de compra e venda	Implementada (com erros de execução)
8	Autenticar um cliente	Implementada (com erros de execução)
9	Consultar saldo	Implementada (com erros de execução)
10	Desconectar cliente	Implementada (com erros de execução)
11	Encerrar plataforma	Implementada (com erros de execução)

9. Conclusão

Este trabalho prático permitiu explorar diversas técnicas de comunicação entre processos, incluindo o uso de named pipes e memória partilhada, além de mecanismos de sincronização como eventos e mutexes. A implementação destes conceitos não apenas reforçou a compreensão teórica, mas também destacou os desafios e soluções práticas na construção de um sistema operativo.

Através da simulação do funcionamento de um mercado de ações, conseguimos criar um ambiente que, embora simplificado, espelha muitos dos processos e interações presentes em sistemas reais. Este projeto demonstrou a importância da coordenação precisa e eficiente entre diferentes componentes de software para alcançar um desempenho robusto e confiável.

Assim, este ampliou a nossa capacidade de desenvolver sistemas operativos complexos e de alta performance, preparando-nos melhor para desafios futuros no campo da engenharia Informática.